

Detailed Explanation of Z-Score in Network Telemetry

1.1 Statistical Background

Z-Score is a statistical technique used to measure how far a data point deviates from the mean of a dataset in terms of standard deviations. It is widely used in anomaly detection because abnormal observations typically deviate significantly from the normal distribution of data.

The Z-Score formula is:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- **X** = observed value
- **μ (mu)** = mean of the dataset
- **σ (sigma)** = standard deviation

The resulting Z-Score indicates how extreme a value is compared to the average behavior of the data.

Typical interpretation of Z-Score values:

Z-Score Range Interpretation

$Z = 0$	Value equals the mean
$0 < Z$	
$2 \leq Z$	

In anomaly detection systems, values where $|Z| > 3$ are commonly considered anomalies.

1.2 Application of Z-Score in Network Telemetry

In network monitoring systems, telemetry data is generated continuously from routers, switches, and servers. These telemetry values form time-series signals representing network behavior over time.

Examples of telemetry metrics include:

- Packet sizes
- Flow duration
- Bytes transmitted
- Packet rate
- Connection count

Under normal network conditions, these metrics tend to follow stable statistical patterns. When unusual activity occurs—such as network attacks, congestion, or failures—these patterns change significantly.

The Z-Score method detects such abnormalities by comparing each observation with the statistical distribution of previous observations.

For example:

If the average packet size is **80,000 bytes** and a new observation is **350,000 bytes**, the Z-Score will be very high, indicating abnormal behavior.

1.3 Z-Score with Short Observation Windows

In this project, Z-Score is applied using **short observation windows (10–50 samples)**.

A sliding window approach works as follows:

1. Select the most recent **N samples** (e.g., 20).
2. Compute the mean and standard deviation within that window.
3. Calculate the Z-Score for the next incoming observation.
4. If the Z-Score exceeds a predefined threshold, it is flagged as an anomaly.

Advantages of using short windows:

- Fast computation
- Quick detection of recent anomalies
- Low memory usage
- Suitable for streaming telemetry

However, small windows may also produce more false positives if the data is highly variable.

1.4 Advantages of Z-Score

The Z-Score method has several advantages for telemetry monitoring:

1. Simplicity

Z-Score requires only basic statistical calculations:

- Mean
- Standard deviation

These operations are computationally inexpensive.

2. Real-Time Processing

Because the algorithm is lightweight, it can process incoming telemetry data in real time.

3. Low Memory Requirement

Only a small window of recent observations needs to be stored.

4. Easy Interpretation

Results are easy to understand because anomalies are simply values that deviate significantly from normal behavior.

1.5 Limitations of Z-Score

Despite its advantages, Z-Score also has some limitations.

Sensitivity to Distribution

Z-Score assumes that data roughly follows a normal distribution. If the data distribution is highly skewed, the results may not be accurate.

Sensitivity to Outliers

Extreme values can influence the mean and standard deviation, affecting anomaly detection.

Static Threshold

Using a fixed threshold such as $|Z| > 3$ may not always capture subtle anomalies.

IMPLEMENTATION

LOAD AND SELECT TELEMETRY FEATURE

```
url = "https://raw.githubusercontent.com/defcon17/NSL_KDD/master/KDDTrain+.txt"

columns = [
    "duration", "protocol_type", "service", "flag", "src_bytes", "dst_bytes",
    "land", "wrong_fragment", "urgent", "hot", "num_failed_logins", "logged_in",
    "num_compromised", "root_shell", "su_attempted", "num_root",
    "num_file_creations", "num_shells", "num_access_files", "num_outbound_cmds",
    "is_host_login", "is_guest_login", "count", "srv_count", "serror_rate",
    "srv_serror_rate", "rerror_rate", "srv_rerror_rate", "same_srv_rate",
    "diff_srv_rate", "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
    "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
    "dst_host_serror_rate", "dst_host_srv_serror_rate",
    "dst_host_rerror_rate", "dst_host_srv_rerror_rate", "label", "difficulty"
]

df = pd.read_csv(url, names=columns)

df.head()
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_same_src_port_rate	dst_host_srv_dif
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	0.17	0.03	0.17	
1	0	udp	other	SF	146	0	0	0	0	0	...	0.00	0.60	0.88	
2	0	tcp	private	S0	0	0	0	0	0	0	...	0.10	0.05	0.00	
3	0	tcp	http	SF	232	8153	0	0	0	0	...	1.00	0.00	0.03	
4	0	tcp	http	SF	199	420	0	0	0	0	...	1.00	0.00	0.00	

5 rows x 43 columns

```
data = df["src_bytes"].values
```

Toggle Gemini

SLIDING WINDOW Z-SCORE AND VISUALIZATION

```
1 window = 20
   anomalies = []

   for i in range(len(data)):

       if i < window:
           anomalies.append(0)
           continue

       window_data = data[i-window:i]

       mean = np.mean(window_data)
       std = np.std(window_data)

       z = (data[i] - mean) / std if std != 0 else 0

       if abs(z) > 3:
           anomalies.append(1)
       else:
           anomalies.append(0)

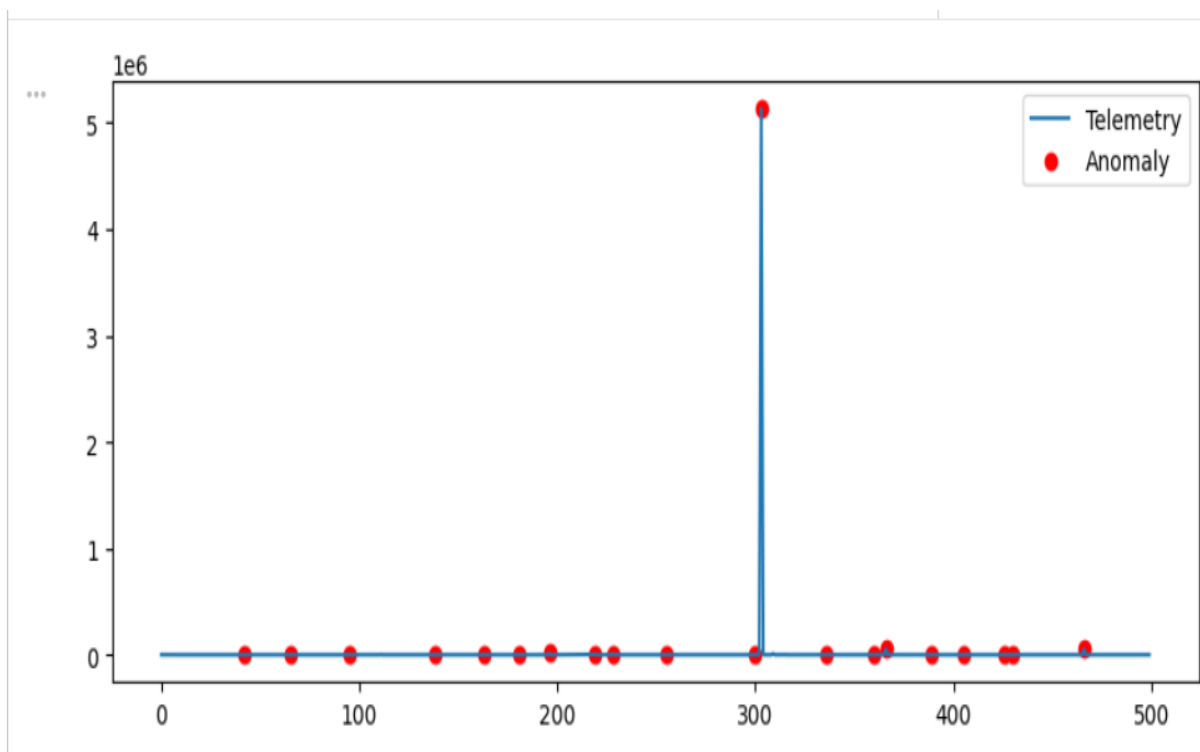
5 plt.figure(figsize=(10,4))

   plt.plot(data[:500], label="Telemetry")

   anomaly_index = np.where(np.array(anomalies[:500])==1)

   plt.scatter(anomaly_index, data[anomaly_index], color='red', label="Anomaly")

   plt.legend()
   plt.show()
```



Z-Score Result

The Z-Score algorithm detected multiple anomalies in the telemetry signal. A significant spike was observed around sample 300, indicating abnormal network activity. This demonstrates the ability of Z-Score to detect sudden deviations in telemetry metrics using statistical thresholds.

Detailed Explanation of Drift Detection Method (DDM)

2.1 Concept of Concept Drift

In real-world streaming systems, the statistical properties of data often change over time. This phenomenon is called concept drift.

Concept drift occurs when:

- Network traffic patterns change
- User behavior changes
- System configuration changes
- Attacks introduce new traffic patterns

For example, a network may experience:

- Normal traffic during the day
- Heavy traffic during peak hours
- Sudden spikes due to attacks

Such changes require adaptive detection systems.

The Drift Detection Method (DDM) is designed to detect these changes.

2.2 Principle of DDM

DDM monitors the error rate of a prediction system over time.

In this project:

- Z-Score identifies anomalies
- Each anomaly detection result forms an error stream

Example error stream:

0 = normal observation

1 = anomaly detected

DDM evaluates whether the error rate increases significantly.

Two parameters are monitored:

- $p_t \rightarrow$ current error rate
- $s_t \rightarrow$ standard deviation of error rate

The algorithm keeps track of the minimum error rate observed so far.

When the error rate increases beyond certain thresholds, the system signals:

- Warning
- Drift

2.3 Drift Detection Thresholds

DDM defines two important conditions.

Warning Level

A warning is triggered when:

$$p_t + s_t \geq p_{min} + 2s_{min}$$

This indicates that the system behavior may be changing.

Drift Level

Drift is detected when:

$$p_t + s_t \geq p_{min} + 3s_{min}$$

At this point, the system concludes that the data distribution has changed significantly.

2.4 Application of DDM in Network Telemetry

In network telemetry monitoring, drift detection helps identify long-term changes in network behavior.

Examples include:

- Gradual increase in network traffic
- Changes in application usage
- Emerging attack patterns
- Network configuration changes

DDM is particularly useful in streaming environments because it continuously monitors the error rate without storing large amounts of data.

13.5 Advantages of DDM

Adaptive Detection

DDM adapts to changing data distributions.

Efficient Streaming Analysis

The algorithm processes data sequentially, making it suitable for real-time telemetry monitoring.

Lightweight Computation

DDM requires only simple statistical calculations and minimal memory.

Early Warning System

The warning stage provides an early signal before a full drift occurs.

2.6 Limitations of DDM

Dependency on Error Stream

DDM relies on the output of another detection system. If the anomaly detection method is inaccurate, DDM results may also be affected.

Slow Detection in Some Cases

Gradual changes in data may take longer to trigger drift detection.

Binary Error Requirement

DDM requires the input data to be converted into binary form (error or correct), which may oversimplify complex telemetry patterns.

IMPLEMENTATION

DDM CODE

```
[24]
✓ 0s errors = np.array(anomalies)

[25]
✓ 0s p_min = float('inf')
s_min = float('inf')

error_sum = 0

warning = []
drift = []

for i in range(len(errors)):

    error_sum += errors[i]

    p = error_sum / (i+1)

    s = np.sqrt(p*(1-p)/(i+1))

    if p + s < p_min + s_min:
        p_min = p
        s_min = s

    if p + s >= p_min + 3*s_min:
        drift.append(i)

    elif p + s >= p_min + 2*s_min:
        warning.append(i)
```

[27]

✓ 2s

```
plt.figure(figsize=(10,4))

plt.plot(errors[:500])

plt.scatter(drift, [1]*len(drift), color="red", label="Drift")

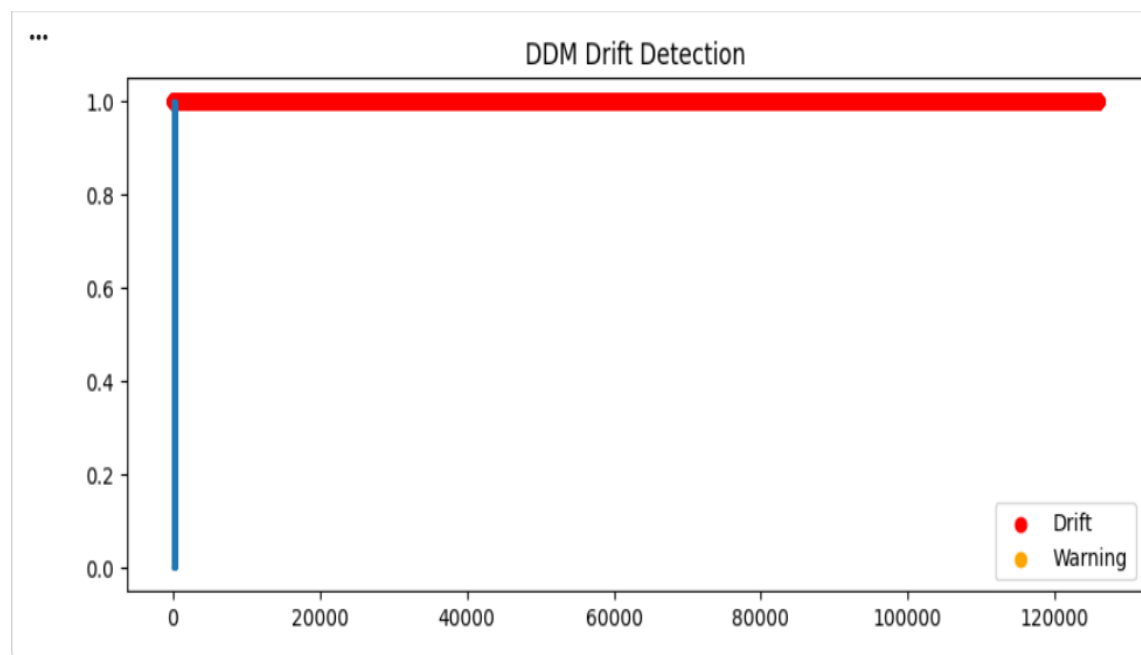
plt.scatter(warning, [1]*len(warning), color="orange", label="Warning")

plt.legend()

plt.title("DDM Drift Detection")

plt.show()
```

VISUALIZATION



DDM Result

The Drift Detection Method (DDM) monitored the error stream produced by the anomaly detection algorithm. Due to the presence of frequent anomalies in the dataset, the algorithm detected multiple drift points. This suggests that the statistical properties of the telemetry data changed over time.

3) Combining Z-Score and DDM

Using Z-Score and DDM together provides two levels of monitoring.

Z-Score performs instant anomaly detection, identifying abnormal values in telemetry metrics.

DDM performs long-term drift detection, identifying changes in the overall data distribution.

The combined workflow works as follows:

1. Network telemetry data is collected.
2. Z-Score detects abnormal observations.
3. An error stream is generated.
4. DDM monitors the error rate.
5. If the error rate increases significantly, drift is detected.

This combination allows the system to detect both:

- short-term anomalies
 - long-term changes in network behavior
-